

Problem S1. Trianglane

This is the same counting idea as Junior J4. Let B be the number of black triangles. Start from $3B$, then subtract 2 for every pair of adjacent black triangles because their common side is not part of the outer boundary.

Horizontal adjacencies are checked inside each row. Vertical adjacencies between the two rows occur only at columns with the correct orientation, which are even columns if columns are numbered from zero.

Complexity. The solution makes a constant number of scans over the input, so it runs in $O(C)$ time.

Problem S2. Symmetric Mountains

For every contiguous crop, its asymmetric value is the sum of absolute differences between pairs of mountains equally far from the two ends. The key recurrence is

$$asym(l, r) = asym(l + 1, r - 1) + |h_l - h_r|.$$

Iterate by center and expand outward. For each center, maintain a running sum; when the interval grows by one mountain on each side, add exactly one new absolute difference. Each interval is generated once, and the minimum value for its length is updated.

This avoids recomputing the whole sum for each interval and turns the naive cubic method into a quadratic one.

Complexity. Expanding around all centers takes $O(N^2)$ time and $O(N)$ memory for the best answer per length.

Problem S3. Palindromic Poster

The task is constructive: produce a grid with exactly R palindromic rows and C palindromic columns, or report that it is impossible. For most cases, a simple pattern works: fill the first R rows and the first C columns with one letter and fill the rest with another. This makes the intended rows and columns symmetric while breaking the others.

The difficult cases occur on the boundaries $R = 0$, $R = N$, $C = 0$, and $C = M$. They can be handled by symmetry: if a column boundary case is easier than a row boundary case, solve the transposed problem and transpose the output back. When all rows must be palindromic, the number of palindromic columns can often be adjusted by changing matching pairs of columns; if the parity prevents the requested number and there is no middle column to use, the instance is impossible.

Small exceptional cases can be solved by direct casework or brute force, while the general construction covers the large cases.

Complexity. Constructing and printing the grid takes $O(NM)$ time.

Problem S4. Minimum Cost Roads

The desired set of roads must preserve the shortest-path structure. For a positive-length edge $e = (a, b, l, c)$, it is useful exactly when it is the unique shortest path from a to b . If a strictly shorter path exists, e is unnecessary. If another path of equal length exists, that equal path can replace e in any optimal solution.

Zero-length edges need special care. First find a minimum spanning forest among the zero-length edges, add its cost, and compress each zero-length connected component into one vertex. Ignore self-loops, and for

multiple edges between the same pair keep the smallest length, breaking ties by cost. Then test the remaining edges using shortest-path distances.

An alternative view is Kruskal-like: process edges in increasing length, breaking ties by cost, and add an edge only if it improves the shortest path between its endpoints.

Complexity. The direct official approach runs Dijkstra from each vertex on the compressed graph, then scans all edges. The exact bound depends on the implementation of shortest paths.

Problem S5. The Filter

The filters have the self-similar structure of the Cantor set. For $N = 3^k$, the valid answers can be generated recursively: the first third and the last third are scaled copies of the previous level, while the middle third is removed.

For general N , use the symmetry of filters and the map

$$f(r) = 3 \min(r, 1 - r).$$

If r is not removed by any filter, then repeated application of f never reaches the first removed interval and eventually cycles among multiples of $1/N$. If r is removed by the k -th filter, then after applying f enough times it reaches the first filter. Thus each fraction x/N can be tested by iterating f with cycle detection or memoization.

For the largest constraints, remove numbers hit by the first several filters, leaving a much smaller candidate set, and then run the iterative test only on those candidates.

Complexity. The intended optimized solution runs in roughly $O(N^{2/3} \log N)$ time.